

CO3 and CO4 Question Bank Answers

Based on the provided question bank and the CO3 and CO4 PPTs.

CO3 Answers

1. Explain the key stages of a CI/CD pipeline and analyze how each stage contributes to automated software delivery.

A CI/CD pipeline is a sequence of automated activities that takes source code from development to deployment. Its main goal is to reduce manual work, detect defects early, and deliver software faster and more reliably.

The common stages are:

1. Source/Version Control

Developers commit code into a shared repository such as GitHub. This creates a single source of truth and allows changes to trigger automation immediately.

2. Build

The application is compiled, packaged, or containerized. Examples include creating a '.jar' file, generating a web build, or building a Docker image. This stage verifies that the code can be transformed into a deployable artifact.

3. Unit Testing

Small isolated pieces of code are tested. This catches logic errors early and gives very fast feedback. Because unit tests are cheap and fast, they are usually run on every commit.

4. Integration Testing

Different modules and services are tested together. This stage helps detect interface mismatches, API issues, database connectivity problems, and configuration errors that unit tests may miss.

5. Security and Quality Checks

Linting, static analysis, dependency checks, secret scanning, image scanning, and SBOM generation may run here. These checks improve maintainability and reduce security risk before deployment.

6. Packaging and Artifact Storage

The verified output is stored in an artifact repository or container registry.

This ensures the exact same version tested in CI is the one later deployed.

7. Staging/Pre-production Deployment

The application is deployed into an environment similar to production. Teams can run smoke tests, performance checks, and final validations here.

8. Production Deployment

In Continuous Delivery, production deployment is prepared and may require approval. In Continuous Deployment, it is automatically released once all checks pass.

How each stage contributes:

- Version control supports collaboration, traceability, and rollback.
- Build makes sure the software is actually runnable.
- Testing increases confidence and reduces production defects.
- Security checks prevent vulnerable or low-quality code from moving forward.
- Packaging provides a repeatable deployable artifact.
- Staging reduces release risk by validating in a realistic environment.
- Deployment automation minimizes human error and speeds up delivery.

Example:

For a Node.js application, a push to GitHub can trigger a pipeline that installs dependencies, runs 'npm test', builds a Docker image, scans it for vulnerabilities, pushes it to a registry, and deploys it to Kubernetes.

In short, CI/CD pipelines convert software delivery from a slow manual activity into a repeatable and reliable automated process.

2. Implement a Git workflow for feature development using branching and pull requests, and explain each step involved.

A good Git workflow for feature development isolates work in a separate branch, allows review through a pull request, and merges only after validation.

Typical workflow:

```
git checkout main
git pull origin main
git checkout -b feature/login-page
```

1. Switch to the main branch

This ensures the developer starts from the latest stable code.

2. Pull the latest changes

This reduces merge conflicts by syncing local code with the remote repository.

3. Create a feature branch

Example: 'feature/login-page'

A separate branch isolates the work and protects the main branch from incomplete code.

4. Develop the feature

Add or modify files as required.

```
git add .  
git commit -m "Add login page UI and validation"
```

5. Stage and commit changes

'git add' places modified files in the staging area.

'git commit' creates a snapshot with a meaningful message.

```
git push origin feature/login-page
```

6. Push the branch to GitHub

This publishes the branch so others can review it.

7. Create a Pull Request

On GitHub, open a PR from 'feature/login-page' into 'main' or 'develop'. Team members then review the code, run checks, and suggest changes if needed.

8. Run PR checks

The PR should pass automated tests, linting, build checks, and security scans. This prevents broken code from being merged.

9. Review and approval

Reviewers validate correctness, readability, and maintainability.

10. Merge the PR

Once approved, the branch is merged into the target branch.

```
git checkout main  
git pull origin main
```

11. Update local main after merge

This keeps the local repository current.

Why this workflow is effective:

- Branching isolates unfinished work.
- Pull requests create a structured review process.
- Automated checks improve code quality.
- The main branch stays stable and deployable.

Example:

If a team is adding payment support, one developer can work in 'feature/payment-gateway' while another works in 'feature/order-history'. Both can open PRs independently without disturbing production-ready code.

3. Compare Git Flow, GitHub Flow, and Trunk-Based Development strategies with suitable examples.

These three strategies differ mainly in branch structure, release style, and speed of integration.

Git Flow

Git Flow uses multiple long-lived and short-lived branches:

- 'main' or 'master' for production
- 'develop' for ongoing integration
- 'feature/*' for new work
- 'release/*' for release preparation
- 'hotfix/*' for urgent production fixes

Best for:

Projects with planned release cycles, large teams, and strong release control.

Example:

An enterprise banking app may develop new features in 'feature/*', merge them into 'develop', create 'release/v2.0' for final testing, and use 'hotfix/*' if an urgent bug appears in production.

Advantages:

- Very structured
- Good for multiple release versions
- Clear separation of development and production

Disadvantages:

- More complex to manage
- Slower for fast-moving teams

GitHub Flow

GitHub Flow is simpler:

- 'main' stays deployable
- work happens in short-lived 'feature/*' branches
- pull requests are used for review and merge

Best for:

Teams practicing continuous delivery or frequent deployments.

Example:

A startup building a web app creates 'feature/search-filter', opens a PR to 'main', runs checks, gets approval, and merges the same day.

Advantages:

- Easy to understand
- Encourages fast iteration
- Fits GitHub-based collaboration well

Disadvantages:

- Less suitable for managing multiple release branches
- Requires strong automated testing because 'main' is updated frequently

Trunk-Based Development

In Trunk-Based Development, developers integrate very frequently into one main branch called 'main' or 'trunk'. Feature branches are very short-lived, sometimes lasting only a few hours.

Best for:

High-velocity teams with mature CI, automated testing, and strong engineering discipline.

Example:

A SaaS platform team commits small changes to 'main' several times per day using feature flags to hide unfinished functionality from users.

Advantages:

- Fast integration
- Fewer long-running merge conflicts
- Excellent fit for continuous integration

Disadvantages:

- Requires excellent test automation
- Developers must avoid large unmerged branches

Comparison Summary

- Git Flow: best for structured release management
- GitHub Flow: best for simple, PR-based continuous delivery
- Trunk-Based Development: best for rapid integration and mature CI/CD

If the project has scheduled releases, Git Flow is useful. If the team wants simplicity, GitHub Flow is ideal. If the team deploys very frequently and has strong automation, Trunk-Based Development is the strongest choice.

4. Develop a GitHub Actions workflow triggered by a push event and explain how jobs, steps, and runners automate CI/CD.

GitHub Actions automates CI/CD using workflow files written in YAML and stored in '.github/workflows/'.

Example workflow:

```
name: Node CI

on:
  push:
    branches:
      - main

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci
```

- name: Run tests

```
run: npm test
```

Explanation:

- 'name' gives the workflow a readable label.
- 'on: push' means the workflow starts whenever code is pushed to the specified branch.
- 'jobs' defines one or more independent units of work.
- 'runs-on' selects the runner environment.

Key concepts:

1. Workflow

A workflow is the complete automation process. A repository can have multiple workflows, such as one for testing and another for deployment.

2. Event

An event triggers the workflow. Common events are 'push', 'pull_request', and 'release'.

3. Job

A job is a group of steps executed on the same runner. Multiple jobs can run in parallel unless dependencies are defined.

4. Step

A step is a single action inside a job. It may run a shell command or use a predefined GitHub Action.

5. Runner

A runner is the machine that executes the job. GitHub provides runners for Linux, Windows, and macOS.

How automation happens:

- Developer pushes code
- GitHub detects the 'push' event
- A runner is allocated
- Steps execute automatically: checkout, setup, install, test
- If any step fails, the job fails and developers are notified

This automates CI/CD by ensuring every code change is validated consistently without manual effort.

5. Analyze the importance of the Software Testing Pyramid and

how it improves software quality and efficiency.

The Software Testing Pyramid is a guideline for balancing different kinds of tests:

- Many unit tests at the base
- Fewer integration tests in the middle
- Very few end-to-end tests at the top

This structure is important because different tests offer different trade-offs in speed, cost, and coverage.

Why it matters

1. Faster feedback

Unit tests run in milliseconds and quickly show whether a code change broke basic logic.

2. Lower cost

It is cheaper to maintain many unit tests than a huge number of slow UI or E2E tests.

3. Better defect detection

Unit tests catch logic errors, integration tests catch interface issues, and E2E tests validate complete user journeys. Together they provide layered protection.

4. Supports CI/CD

Since most tests are fast, teams can run them frequently in pipelines without delaying releases.

5. Reduces flaky tests

Over-reliance on E2E tests often leads to unstable pipelines. The pyramid limits that risk.

Layer-wise role

- Unit tests verify individual functions or classes
- Integration tests verify how modules communicate
- E2E tests verify full real-world workflows

Example:

For an online shopping app:

- Unit test: calculate cart total correctly

- Integration test: verify order service talks to payment service
- E2E test: customer logs in, adds items, pays, and receives confirmation

How it improves quality and efficiency:

- Bugs are caught earlier
- Developers spend less time debugging late-stage failures
- Pipelines remain faster
- Teams gain confidence to release more often

So, the testing pyramid improves both software quality and delivery speed by encouraging the right proportion of tests at each level.

6. Compare the unit build stage and container build stage with examples, and explain their role in DevOps workflows.

The unit build stage and container build stage are both important, but they solve different problems.

Unit Build Stage

Purpose:

To verify the correctness of small units of code using automated tests.

What happens:

- Source code is compiled if needed
- Unit tests run on functions, methods, or classes
- Failures stop the pipeline early

Example:

In Java, a method 'add_numbers(2, 3)' should return '5'. A JUnit test checks this automatically.

Role in DevOps:

- Detects coding defects early
- Gives fast feedback to developers
- Prevents broken code from progressing through the pipeline

Container Build Stage

Purpose:

To package the application and its dependencies into a container image.

What happens:

- Docker reads instructions from a 'Dockerfile'
- Base image, dependencies, source code, and runtime command are assembled
- Final image is created and pushed to a registry

Example Dockerfile:

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Role in DevOps:

- Produces a consistent deployable artifact
- Ensures the same environment across development, testing, and production
- Supports cloud-native deployment using Kubernetes or Docker platforms

Comparison

- Unit build stage focuses on correctness
- Container build stage focuses on packaging and portability
- Unit stage outputs test results
- Container stage outputs a deployable image

In a DevOps workflow, the unit build stage usually comes before the container build stage. First we prove the code works, then we package it for deployment.

7. Implement Docker layer caching techniques and explain how Dockerfile instruction ordering improves build performance.

Docker creates image layers for many instructions in a 'Dockerfile'. If a layer has not changed, Docker can reuse it from cache instead of rebuilding it.

A well-optimized Dockerfile:

```
FROM node:20
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci
```

```
COPY . .  
RUN npm run build  
  
CMD ["npm", "start"]
```

Why this is good:

- Dependency files are copied before source code
- 'npm ci' runs in an earlier layer
- If only application source changes, Docker reuses the dependency layer

Bad ordering:

```
FROM node:20  
WORKDIR /app  
COPY . .  
RUN npm ci  
RUN npm run build
```

Here, even a small source code change invalidates the layer cache for 'npm ci', causing dependencies to be installed again.

Techniques for Docker layer caching:

1. Order stable steps first

Put expensive but rarely changing steps early.

2. Put frequently changing steps later

This minimizes cache invalidation.

3. Use '.dockerignore'

Exclude files like 'node_modules', logs, and temporary files so unnecessary changes do not break the cache.

4. Use BuildKit cache mounts

```
RUN --mount=type=cache,target=/root/.npm npm ci
```

This persists package manager cache across builds.

5. Use external cache in CI/CD

Tools such as Docker Buildx can export and import cache between pipeline runs.

Result:

Build times drop significantly because Docker skips repeated work and rebuilds only the layers affected by changes.

8. Analyze different cache optimization strategies in Docker builds and their impact on CI/CD efficiency.

Docker cache optimization strategies help reduce build time, lower CI costs, and improve developer productivity.

1. Layer Ordering

Place rarely changing instructions earlier and frequently changing ones later.

Impact:

Reduces unnecessary rebuilds and speeds up repeated pipelines.

2. Small Build Context

Use '.dockerignore' to keep the build context small.

Example:

Exclude 'node_modules', '.git', logs, and temporary files.

Impact:

- Faster transfer to Docker daemon
- Less chance of cache invalidation
- Lower storage overhead

3. Cache Mounts

Package manager caches can be preserved:

```
RUN --mount=type=cache,target=/root/.npm npm ci
```

Impact:

Dependencies are not downloaded from scratch in every build.

4. External/Remote Cache

CI environments are often temporary. Remote cache allows one build to reuse layers created by a previous build.

Example:

‘docker buildx build --cache-from ... --cache-to ...‘

Impact:

- Faster builds across pipeline runs
- Better efficiency in cloud CI systems
- Reduced compute cost

5. Multi-stage Builds

Use one stage for compilation and another for runtime.

Impact:

- Smaller final images
- Less attack surface
- Faster downstream pull and deploy times

6. Dependency Isolation

Copy only dependency manifest files first, install packages, then copy application source.

Impact:

Source changes do not force dependency reinstall.

Overall impact on CI/CD:

- Shorter pipeline duration
- Faster feedback to developers
- Lower infrastructure cost
- More frequent and reliable releases

So, cache optimization is not just a performance trick; it directly improves CI/CD productivity and delivery speed.

9. Explain the process of generating an SBOM using tools like Syft and analyzing vulnerabilities using Grype.

An SBOM, or Software Bill of Materials, is a structured inventory of all components inside an application or container image. It helps teams understand what software is included and identify vulnerable dependencies.

Step 1: Build or select the image

Assume the image is:

```
docker build -t my-node-app .
```

Step 2: Generate the SBOM using Syft

```
syft my-node-app -o json > sbom.json
```

What Syft does:

- Scans the image or filesystem
- Lists packages, libraries, versions, and metadata
- Produces output in formats such as JSON or CycloneDX

Step 3: Analyze vulnerabilities using Gripe

```
gripe sbom:sbom.json
```

What Gripe does:

- Reads the SBOM
- Compares listed packages against known vulnerability databases
- Reports CVE ID, severity, affected package, and possible fixes

Step 4: Review and act

The team reviews high and critical issues first, updates packages, rebuilds the image, and scans again until the risk is acceptable.

Example:

If the SBOM shows an outdated OpenSSL package and Gripe reports a high-severity CVE, the base image or package version should be updated immediately.

Why this process is useful:

- Provides visibility into image contents
- Speeds up vulnerability response
- Improves compliance and audit readiness
- Strengthens software supply chain security

10. Examine the role of vulnerability scanning and SBOM generation in ensuring container security and reliability.

Vulnerability scanning and SBOM generation are complementary practices in container security.

Role of vulnerability scanning

Vulnerability scanning checks images for known security issues in operating system packages, libraries, and application dependencies.

Tools:

- Trivy
- Grype

Benefits:

- Detects known CVEs before deployment
- Prevents insecure images from reaching production
- Supports policy enforcement in CI/CD

Role of SBOM generation

SBOM generation creates a full inventory of image contents.

Tools:

- Syft
- CycloneDX-based tools

Benefits:

- Shows what components exist inside the image
- Supports dependency tracking
- Helps teams understand exposure when a new CVE is announced

Why both are needed

- The SBOM tells us what is inside the image
- The scanner tells us what is wrong with those components

Example:

An image may contain an old 'openssl' package. The SBOM lists it clearly, and the vulnerability scanner detects that the version is affected by a serious CVE.

Impact on reliability:

- Reduces failures caused by insecure or unsupported packages
- Improves trust in deployed images
- Helps standardize secure release practices

In CI/CD pipelines, these checks should run automatically during build or pre-release stages so that risky containers are blocked before deployment.

CO4 Answers

1. Explain the role of IAM and RBAC in Kubernetes and how they help secure cluster resources.

IAM, or Identity and Access Management, controls who can access which resources and what actions they are allowed to perform. In Kubernetes, this security model works together with RBAC, which stands for Role-Based Access Control.

IAM role in Kubernetes

IAM handles:

- Authentication: verifying who the user or service is
- Authorization foundation: connecting that identity to permission rules

A typical access flow is:

1. User or service sends a request
2. Kubernetes API server authenticates the identity
3. RBAC checks whether that identity is allowed to perform the action
4. Admission controllers apply additional policies
5. Resource access is allowed or denied

RBAC role in Kubernetes

RBAC defines permissions using:

- 'Role' or 'ClusterRole'
- 'RoleBinding' or 'ClusterRoleBinding'
- 'ServiceAccount'

A role specifies allowed actions such as 'get', 'list', 'create', or 'delete' on resources like pods, services, or secrets.

Example:

A developer may be allowed to view pods in one namespace but not read secrets or delete deployments.

How IAM and RBAC secure the cluster:

- Prevent unauthorized access
- Enforce least privilege
- Reduce accidental or malicious changes
- Separate duties between developers, operators, and applications

Without proper IAM and RBAC, any user or pod could potentially access sensitive data or modify important workloads, which is a major security risk.

2. Demonstrate how Kubernetes Secrets are created, managed, and used securely within applications.

Kubernetes Secrets store sensitive data such as passwords, tokens, API keys, and certificates. They are safer than putting such values directly in application YAML files or source code.

Step 1: Encode the value in Base64

```
echo -n "mydbpassword" | base64
```

Example output:

```
bXlkYnBhc3N3b3Jk
```

Step 2: Create a Secret manifest

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  password: bXlkYnBhc3N3b3Jk
```

Step 3: Apply it to the cluster

```
kubectl apply -f secret.yaml
kubectl get secrets
```

Step 4: Use the Secret in a pod

As an environment variable:

```
env:  
- name: DB_PASSWORD  
valueFrom:  
secretKeyRef:  
name: db-secret  
key: password
```

Or as a mounted file:

```
volumes:  
- name: secret-volume  
secret:  
secretName: db-secret
```

Step 5: Confirm access

```
kubectl get pod  
kubectl describe pod <pod-name>
```

Management and security practices:

- Restrict access using RBAC
- Avoid storing plaintext secrets in Git
- Rotate secrets regularly
- Use namespace isolation
- Enable encryption at rest for etcd when possible

Example:

A web app that connects to a database can read the password from a Secret at runtime, instead of hard-coding it in 'deployment.yaml'.

3. Compare Kubernetes Secrets and Sealed Secrets in terms of security, usability, and GitOps compatibility.

Kubernetes Secrets and Sealed Secrets both store sensitive information, but they differ greatly in how safely they can be handled in version-controlled workflows.

Kubernetes Secrets

Characteristics:

- Stored as Base64-encoded data
- Can be protected by RBAC
- Not safe to commit directly to Git in raw form

Security:

Base64 is not encryption. Anyone with access can decode it easily.

Usability:

Simple to create and use directly with 'kubectl'.

GitOps compatibility:

Weak, because storing plaintext or Base64 secrets in Git is risky.

Sealed Secrets

Characteristics:

- Secret data is encrypted using a public key
- Only the controller inside the cluster can decrypt it

Security:

Much stronger for Git-based workflows because encrypted data can be stored safely in repositories.

Usability:

Slightly more setup is required because the Sealed Secrets controller and sealing process must be used.

GitOps compatibility:

Excellent, because teams can commit encrypted secrets alongside deployment manifests.

Comparison Summary

- Secrets: simple, direct, less secure in Git workflows
- Sealed Secrets: secure, GitOps-friendly, slightly more complex

Example:

If a team uses Argo CD or Flux and stores all Kubernetes manifests in Git, Sealed Secrets are the better choice because they protect credentials throughout the CI/CD pipeline.

4. Apply rollout and rollback strategies in Kubernetes and analyze their importance in minimizing deployment risks.

In Kubernetes, a rollout means deploying a new application version, while a rollback means returning to a previous stable version when something goes wrong.

Rollout

Kubernetes Deployments usually use the 'RollingUpdate' strategy.

Example command:

```
kubectl set image deployment/myapp mycontainer=myapp:v2  
kubectl rollout status deployment/myapp
```

This gradually replaces old pods with new ones while trying to keep the service available.

Useful settings:

- 'maxUnavailable': how many old pods can be unavailable during update
- 'maxSurge': how many extra pods can be created temporarily

Rollback

If the new version fails due to crash, bad configuration, or readiness issues:

```
kubectl rollout undo deployment/myapp  
kubectl rollout history deployment/myapp
```

This restores a previous working revision.

Why rollout and rollback are important

- Reduce downtime
- Limit the blast radius of bad releases
- Allow quick recovery
- Improve confidence in frequent deployments

Example:

Suppose version 'v2' of an application fails readiness probes and users cannot access the service. Kubernetes pauses the rollout, and operators can

immediately roll back to 'v1'.

Best practices:

- Use rolling updates in production
- Monitor logs and metrics during rollout
- Avoid 'latest' image tags
- Keep revision history
- Test rollback procedures in advance

These practices minimize deployment risk and keep services stable.

5. Explain the concept of admission controllers and their role in enforcing security and governance policies.

Admission controllers are Kubernetes components that inspect requests after authentication and authorization but before the object is stored in the cluster.

They act like gatekeepers.

They can:

- Allow a request
- Reject a request
- Modify a request

Main types

1. Mutating Admission Controllers

These change or add configuration before the object is accepted.

Example:

If a pod is created without CPU and memory limits, a mutating controller can automatically insert default values.

2. Validating Admission Controllers

These check whether the request follows rules but do not modify it.

Example:

A policy may reject images using the 'latest' tag or pods running in privileged mode.

Security and governance role

Admission controllers help enforce:

- Pod security rules
- Resource quotas
- Standard labels and annotations
- Image policy rules
- Organization compliance requirements

Example scenarios:

- Reject a pod running as root
- Reject a deployment exceeding namespace CPU quota
- Automatically add default limits to maintain cluster stability

Their role is important because they ensure every object entering the cluster follows required policies, even if developers make mistakes in their YAML files.

6. Analyze the use of observability tools such as logs and node metrics in troubleshooting Kubernetes clusters.

Lightweight observability in Kubernetes uses simple built-in tools to understand cluster behavior without a heavy monitoring stack.

The three common signals are:

- Logs
- Metrics
- Events

Logs

Command:

```
kubectl logs <pod-name>
```

Logs help identify application-level errors such as:

- database connection failure
- port binding errors
- missing configuration

Example:

'kubectl logs webapp-pod' may show 'Error: Database connection failed'.

Node metrics

Command:

```
kubectl top node
```

Node metrics help identify:

- overloaded nodes
- memory pressure
- cluster capacity problems

Example:

If one node shows 95% CPU usage, it may be causing slow scheduling or degraded performance.

Pod metrics

Command:

```
kubectl top pod
```

These metrics help detect which workload consumes too much CPU or memory.

Events

Command:

```
kubectl get events
```

Events explain state changes such as scheduling failures, image pull issues, or container restarts.

Why these tools are useful:

- Quick troubleshooting
- No complex setup
- Good for small clusters and development environments

Limitations:

- No long-term storage

- Less detailed than Prometheus/Grafana

Still, logs and metrics are often enough to diagnose common operational problems quickly.

7. Examine common Kubernetes failure scenarios (e.g., CrashLoopBackOff) and explain how they can be diagnosed.

Kubernetes failures often show up as pod states, events, or unhealthy rollout behavior. Diagnosis usually begins with 'kubectl'.

1. CrashLoopBackOff

Meaning:

The container starts, crashes, restarts, and repeats.

Diagnosis:

```
kubectl get pods
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

Common causes:

- bad environment variables
- missing database connection
- application startup failure
- invalid configuration

2. ImagePullBackOff

Meaning:

Kubernetes cannot pull the container image.

Diagnosis:

```
kubectl describe pod <pod-name>
```

Common causes:

- wrong image name
- missing tag
- private registry credentials not configured

3. Pending Pod

Meaning:

Pod cannot be scheduled.

Diagnosis:

```
kubectl describe pod <pod-name>  
kubectl get events
```

Common causes:

- insufficient CPU or memory
- missing node selectors or tolerations
- unbound persistent volume claim

4. High Resource Usage

Diagnosis:

```
kubectl top node  
kubectl top pod
```

Common causes:

- memory leaks
- oversized traffic load
- missing resource limits

5. Failed Rollout

Diagnosis:

```
kubectl rollout status deployment/myapp  
kubectl describe deployment myapp
```

Common causes:

- readiness probe failure
- image issues
- bad application version

Example:

If a pod is in 'CrashLoopBackOff', checking previous logs may reveal 'Database

connection failed', indicating the app cannot reach the database or the secret is incorrect.

8. Apply RBAC principles using roles, role bindings, and service accounts to enforce least-privilege access.

RBAC implements least privilege by granting only the minimum permissions needed for a user or workload.

Example Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

This role allows only read operations on pods in the 'dev' namespace.

Example ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: dev
```

Example RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

How this enforces least privilege:

- The service account can only view pods
- It cannot delete deployments
- It cannot read secrets unless explicitly granted

Why this matters:

- Limits damage from compromised pods
- Prevents accidental misuse
- Improves auditability

Best practices:

- Prefer namespace-scoped 'Role' over 'ClusterRole' when possible
- Avoid broad permissions like '*'
- Use separate service accounts for different applications
- Review bindings regularly

This is a practical and secure way to control access in Kubernetes.

9. Analyze cost optimization techniques in Kubernetes clusters and their impact on performance and efficiency.

Kubernetes can become expensive when workloads request too many resources, nodes remain underutilized, or unused resources stay active.

Key cost optimization techniques

1. Right-size requests and limits

Set CPU and memory values based on actual usage rather than guesswork.

Impact:

Reduces over-provisioning and improves node utilization.

2. Monitor pod and node usage

```
kubectl top pod  
kubectl top node
```

Impact:

Helps identify waste, overloaded pods, and idle nodes.

3. Scale based on demand

Use HPA or manual scaling to increase pods during peak traffic and reduce them

during low traffic.

Impact:

Matches cost to workload demand.

4. Clean unused resources

Remove unused namespaces, orphaned services, idle pods, and old volumes.

Impact:

Eliminates hidden recurring cost.

5. Manage storage carefully

Delete unused PVCs and choose suitable storage classes.

Impact:

Prevents paying for unnecessary storage capacity.

Performance trade-off

Cost optimization should not become under-provisioning.

If requests are set too low:

- pods may be throttled
- performance may drop
- instability may increase

So the goal is not simply to reduce resources, but to right-size them.

Example:

If a pod consistently uses only 200m CPU but requests 1000m CPU, lowering the request can free space for other workloads and may allow the cluster to run on fewer nodes.

In small clusters, these simple optimization practices provide major savings without requiring advanced commercial tooling.

10. Analyze how Architecture Decision Records (ADRs) improve maintainability, traceability, and decision-making in Kubernetes-based systems.

An Architecture Decision Record, or ADR, is a short document that captures:

- the context of a problem
- the decision taken

- the status
- the consequences

In Kubernetes-based systems, teams make many recurring decisions about deployment style, security controls, observability, and cost management. If these choices are not documented, future team members may not understand why the system was designed in a particular way.

How ADRs improve maintainability

- They preserve architectural knowledge
- They reduce confusion when team members change
- They help future engineers maintain the system without repeating old mistakes

How ADRs improve traceability

- They record why a decision was made
- They connect system behavior back to a documented reason
- They make audits and reviews easier

How ADRs improve decision-making

- Teams can compare new proposals against past reasoning
- Superseded decisions remain visible instead of being forgotten
- Discussions become more objective because context is documented

Example ADR topic:

?Use Kubernetes Secrets with Sealed Secrets for GitOps-managed credentials.?

Possible content:

- Context: plain YAML secrets are risky in Git
- Decision: use Sealed Secrets
- Consequences: safer Git storage, extra controller dependency

Best practices:

- Keep ADRs short and clear
- Write them when the decision is made
- Store them in the repository, such as '/docs/adr/'
- Do not delete old ADRs; mark them as superseded

Therefore, ADRs improve long-term system quality by making important architecture choices visible, understandable, and reviewable.